

期末報告: 2048 閃電進化版

姓名： 林恩廷 學號： 11428105

一、 專案簡介

- **專案目的：** 專案目的在製作經典益智遊戲「2048」，並透過加入「時間壓迫機制」、「特殊方塊（黃金方塊、萬能史萊姆）」以及「時空回溯」等創新玩法，提升遊戲的策略性與豐富度。
- **主要功能：**
 1. 支援三種難度選擇（20秒、45秒、60秒）與動態時間進度條。
 2. 完整的 WASD 滑動與方塊數值合併演算法。
 3. 分數計算與最高紀錄（PR）檔案讀寫保存。
 4. 支援到達50 步的悔棋功能。
- **使用語言：** C 語言 (搭配 `rlutil.h` 跨平台終端機標頭檔)

二、 開發過程

本專案的開發過程分為四個主要階段，從概念發想一路到跨平台最佳化，一步步將功能疊加：

1. **概念發想與玩法設計：** 確定專案目標不只侷限於傳統 2048，而是要增加遊戲的刺激感與容錯率。一開始發想出「倒數計時模式」來製造壓迫感，接著為了平衡難度，設計了消耗分數的「時空回溯」機制，最後引入機率生成的「黃金方塊」與「萬能史萊姆」，大幅提升策略深度與趣味性。
2. **基礎架構與 UI 建立：** 一開始匯入 `rlutil.h` 函式庫，用終端機文字定位與高對比純色渲染，打造出無閃爍且極具現代感的遊戲介面。
3. **遊戲核心邏輯實作：** 2048 最複雜的部分在於上下左右滑動時的方塊合併判斷，為了避免程式碼寫得太過冗長混亂，我請 AI 協助優化移動邏輯，將四個方向的移動簡化成統一的處理流程，讓方塊的滑動與相加在畫面上表現得非常流暢，且大幅降低了卡 Bug 的機率。
4. **進階玩法與視覺實作：** 我將自己發想的「特殊方塊」與「悔棋」功能具現化，透過與 AI 的來回討論，順利在遊戲畫面上加入了亮眼的「黃金方塊」、紫色的「萬能史萊姆」，以及會隨著時間縮短的動態進度條，讓原本單調的數字遊戲變得豐富且充滿視覺回饋。

三、 AI 使用方式

在本次專案中，我將 AI 視為「程式碼除錯幫手」與「演算法助教」：

- **規劃模式：** 遊戲初期請 AI 協助拆解開發步驟，決定先想出遊戲邏輯在開始寫程式。
- **編程模式：** 利用 AI 協助生成繁瑣的 `switch-case` 顏色映射表，以及撰寫檔案讀寫（`fopen, fprintf`）的基礎樣板程式碼。
- **問答與除錯：** 遇到終端機亂碼、畫面破圖或編譯器報錯時，將終端機畫面的截圖或錯誤代碼提供給 AI，請其分析原因並給出解決方案。

四、 提示詞範例與互動過程

以下列出幾個在開發過程中，最具代表性的提示詞與除錯協作過程：

- **提示詞 1（網頁終端機除錯：畫面閃爍與顏色破圖）：** 「在學校網站執行時，選單畫面會一直閃爍，而且方塊的背景顏色出不來、排版也被拉長變形」

- **AI 回應與採用：** AI 分析指出，網頁版終端機對背景填色（`setBackgroundColor`）的支援度極差，容易將顏色吃掉變成隱形，且頻繁刷新會產生 Buffer 緩衝延遲。AI 協助我重寫渲染邏輯：捨棄背景填色改用「高對比霓虹文字邊框」、將方塊字串長度嚴格鎖死為 8 個字元以防止跑位，並在每次繪製後加入 `fflush(stdout)` 強制推播畫面，成功解決了嚴重的閃爍與顏色破圖問題。
- **提示詞 2（網頁高度限制與按鍵干擾排解）：** 「遊戲畫面的最下面會被吃掉，而且按方向鍵會跑出奇怪的亂碼，導致方塊自己亂滑動。」
 - **AI 回應與採用：** AI 發現網頁終端機的高度限制（約 20 行）會導致畫面強制捲動而崩潰，且線上環境對實體方向鍵的解碼有 Bug（會產生 `^[A` 等亂碼干擾程式），在 AI 的建議下，將 UI 總高度極致壓縮至 16 行內，並將玩家操作嚴格綁定於 `W`、`A`、`S`、`D` 字母鍵，徹底排除了網頁環境的干擾，讓遊戲達到 100% 的穩定流暢。

五、問題與解決方法

- **問題 1：結算畫面被亂按跳過。**
 - **描述：** 遊戲結束時，若玩家死前狂按 WASD，殘留在「鍵盤緩衝區」的輸入會直接觸發結算畫面的「按任意鍵離開」，導致畫面瞬間全黑。
 - **解決方法：** 加入迴圈 `while(kbhit()) getkey();` 清空緩衝區，並寫死判定，強制要求玩家只能按下 `Space` 或 `Enter` 才能退出結算畫面。
- **問題 2：Linux 網頁終端機「方向鍵」引發不規則移動。**
 - **描述：** 網頁版環境對方向鍵的處理有 Bug，會解碼成 `^[A` 等亂碼字元，導致程式誤判。
 - **解決方法：** 在 `processMove` 函式中，嚴格將操作綁定在實體字母鍵 `w`、`a`、`s`、`d` 上，阻擋其他雜訊干擾。

六、成果展示

以下為遊戲實際執行畫面

1. 難度選擇主選單



說明：一鍵選擇難度，無閃爍的 UI 設計。

2. 遊戲進行畫面

```

• Terminal

=== 2048 LIGHTNING EDITION ===

Score: 216
Time : 56 / 60 sec
[===== ]

[ 16 ] [ 2 ] [ 8 ] [   ]
PR   : 216

[ 4 ] [ 32 ] [   ] [ ?? ]

--- Special Tiles ---
[ *N* ] Golden Tile
-> 3x Score & +5 Sec

[ 4 ] [ 2 ] [   ] [   ]
[ ?? ] Slime Tile
-> Merge w/ ANYTHING

=====
Controls: [W A S D] Move | [U] Undo
          [Q] Quitw [C^][D

```

```
• Terminal

=== 2048 LIGHTNING EDITION ===

Score: 36
Time : 41 / 60 sec
[===== ]

[      ] [      ] [      ] [      ]
PR    : 36

[      ] [      ] [  4  ] [ * 2* ]

--- Special Tiles ---
[      ] [ * 2* ] [      ] [  8  ] [ *N* ] Golden Tile
-> 3x Score & +5 Sec

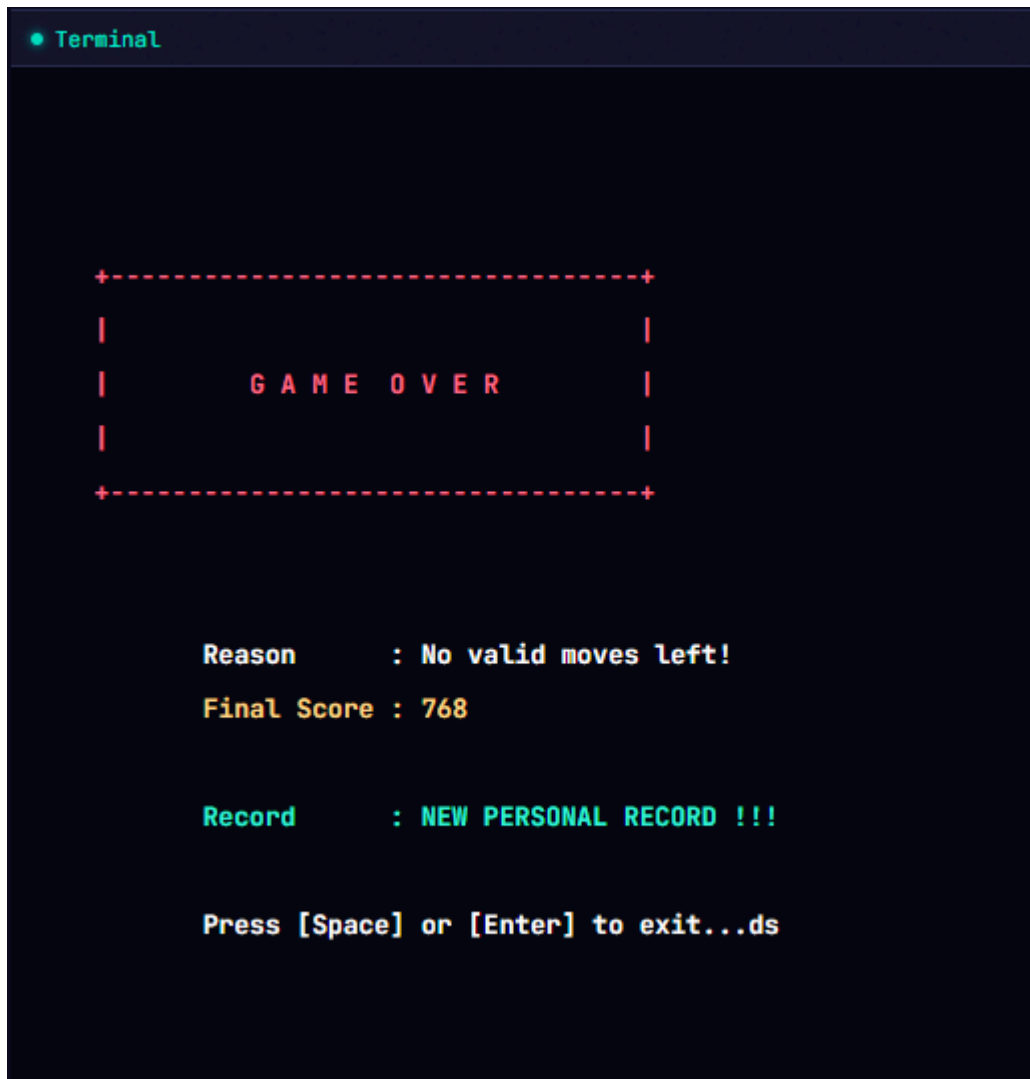
[      ] [  4  ] [  8  ] [  2  ] [ ?? ] Slime Tile
-> Merge w/ ANYTHING

=====

Controls: [W A S D] Move | [U] Undo
          [Q] Quits [C^][D]
```

說明：右側具有會隨時間扣除的血條式進度條，盤面上有精準對齊的數字方塊與特殊道具。

3. 結算與 PR 紀錄畫面



```
Terminal

+-----+
|               |
|   G A M E O V E R   |
|               |
+-----+

Reason      : No valid moves left!
Final Score : 768

Record      : NEW PERSONAL RECORD !!!

Press [Space] or [Enter] to exit...ds
```

說明：自製的文字邊框結算畫面，清楚顯示死因、最終分數，並讀取檔案判斷是否突破個人歷史最高紀錄 (PR)。

七、學習心得與反思

在本次專案開發中，生成式 AI 大幅縮短了我卡底層 Bug 的時間，讓我能將更多精力集中在遊戲邏輯的建構與玩法創新上，因為我是先透過 VSC 把專案做好，不知道會有不同系統的差異，透過與 AI 的除錯過程，我學到了許多課堂上較少深入討論的實務經驗，例如不同作業系統底層渲染的差異、終端機緩衝區的概念，這些知識不僅解決了眼前的問題，更拓展了我對 C 語言跨平台執行的理解。

在協作過程中，我也了解到 AI 作為輔助工具的侷限性，AI 無法直接看到程式執行的動態畫面，當終端機出現畫面閃爍、排版，或是因為網頁版方向鍵亂碼導致的幽靈移動時，這要求我必須具備良好的問題追蹤與描述能力，透過提供精準的描述、還原操作步驟，才能精確引導 AI 找出真正的 Bug 核心。這段歷程讓我明白，無論 AI 多麼強大，開發者的觀察力與邏輯能力，也是解決問題不可或缺的關鍵。

回顧整個專案，雖然目前已順利達成預期的核心功能與特殊玩法，並成功克服了線上環境的挑戰，但專案仍有許多可以持續優化的空間。例如，目前的悔棋功能受限於陣列大小，最多只能回溯 50 步，未來我希望能嘗試將歷史紀錄寫入二進位檔案中，實現無限步數回溯的機制。

八、AI 使用聲明

- **由我自己獨立完成的部分：** 專案整體的玩法企劃、邏輯驗證、測試除錯、遊戲數值平衡調整（特殊方塊機率設計）、報告撰寫與截圖排版。
- 接受 AI 協助的部分：
 1. **演算法架構優化：** 在處理 2048 核心的滑動與合併邏輯時，AI 協助實作了「一維陣列降維」的演算法，大幅簡化了原本需要分四個方向獨立判斷的冗長迴圈，使程式碼更為精簡高效。
 2. **跨平台 UI 渲染與除錯：** AI 協助處理了 `rlutil.h` 的高對比色彩配置，在面對學校 Linux 網頁終端機的相容性 Bug 時，AI 幫我找出了緩衝區延遲的根本原因，並提供 `fflush(stdout)` 強制推播、捨棄背景填色、精準鎖定字串長度等解決方案，成功修復了破圖與閃爍問題。

九、核心功能程式碼片段與說明

1. 時空回溯系統

此片段利用資料結構中「堆疊 (Stack)」與「滑動窗口」的概念，實作最多 50 步的悔棋機制：

```
void saveState() {
    if (historyCount < MAX_UNDO) {
        history[historyCount++] = game; // 將當前遊戲狀態壓入堆疊
    } else {
        // 當堆疊滿了，將所有歷史紀錄向前覆蓋移一格，保留最新狀態
        for (int i = 0; i < MAX_UNDO - 1; i++) history[i] = history[i+1];
        history[MAX_UNDO - 1] = game;
    }
}
```

2. 特殊方塊機率生成機制

此片段實作了遊戲中「黃金方塊」與「萬能紫色史萊姆」的稀有度與隨機刷出邏輯：

```
void spawnTile() {
    int emptySpots[SIZE * SIZE][2];
    int count = 0;
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (game.grid[i][j] == 0) {
                emptySpots[count][0] = i;
                emptySpots[count][1] = j;
                count++;
            }
        }
    }
    if (count > 0) {
        int randIndex = rand() % count;
        int r = emptySpots[randIndex][0];
        int c = emptySpots[randIndex][1];

        int chance = rand() % 100;
        if (chance < 5) game.grid[r][c] = 1; // 5% 機率生成紫色萬能史萊姆 (數值以 1
```

代表)

```

    else if (chance < 15) game.grid[r][c] = -((rand() % 10 < 9) ? 2 : 4); //
10% 機率生成黃金方塊 (以負數代表)
    else game.grid[r][c] = (rand() % 10 < 9) ? 2 : 4; // 85% 生成普通方塊
    }
}

```

3.一維陣列降維滑動演算法

此片段為 2048 核心的合併演算法，負責處理單一向量上的「靠攏 → 合併 → 靠攏」邏輯：

```

int slideArray(int array[SIZE]) {
    int moved = 0;
    int writePos = 0;
    for (int i = 0; i < SIZE; i++) {
        if (array[i] != 0) {
            if (writePos != i) {
                array[writePos] = array[i];
                array[i] = 0;
                moved = 1;
            }
            writePos++;
        }
    }
    for (int i = 0; i < SIZE - 1; i++) {
        if (array[i] != 0 && array[i + 1] != 0) {
            int v1 = abs(array[i]);
            int v2 = abs(array[i + 1]);
            int isW1 = (v1 == 1);
            int isW2 = (v2 == 1);

            if (v1 == v2 || isW1 || isW2) {
                int mergedVal = 0;
                if (isW1 && isW2) mergedVal = 4;
                else if (isW1) mergedVal = v2 * 2;
                else if (isW2) mergedVal = v1 * 2;
                else mergedVal = v1 * 2;

                int isGolden = (array[i] < 0 || array[i + 1] < 0);
                array[i] = mergedVal;
                array[i + 1] = 0;
                moved = 1;

                if (isGolden) {
                    game.score += mergedVal * 3;
                    timeLeft += 5;
                } else {
                    game.score += mergedVal;
                    timeLeft += 1;
                }
            }
            if (timeLeft > maxTime) timeLeft = maxTime;
        }
    }
}

```

```
        }  
    }  
}  
writePos = 0;  
for (int i = 0; i < SIZE; i++) {  
    if (array[i] != 0) {  
        if (writePos != i) {  
            array[writePos] = array[i];  
            array[i] = 0;  
        }  
        writePos++;  
    }  
}  
return moved;  
}
```